

## Week 7

### R에 있는 기본 그래픽 도구

hist() - Histograms // 히스토그램  
 plot() - Scatterplots, etc. //scatterplot 원이 여러개 있는 모양  
 boxplot() - Box- and whiskerplots

### attach()

attach(데이터셋 이름)  
 attach(exercise) #exercise에 있는 열이름에는 앞으로 exercise\$ 표시를 생략하겠다

### plot()

scatterplot을 그림. 원으로 변수를 표시한 것  
**plot(x축에 들어갈 변수, y축에 들어갈 변수)**  
 plot(exercise\$calories, exercise\$workout\_distance)  
 #그냥 변수이름만 쓰면 어떤 데이터셋에 있는건지 알 수 없음

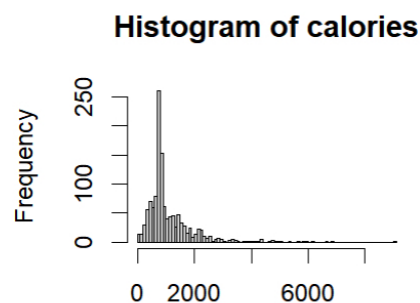
**모양바꾸기 옵션** pch = 숫자  
 미리 정해진 모양으로 모양이 바뀜.  
 plot(calories, workout\_distance, pch=15)

**색바꾸기 옵션** col = '색깔이름'  
 plot(calories, workout\_distance, col="blue", pch=15)

**범위제한 옵션** xlim = c(0,8000)  
 plot(calories, workout\_distance, col="blue", pch=15, xlim=c(0,8000))

### hist()

**히스토그램을 그림**  
 hist(열 변수이름, breaks=숫자) #100개로 쪼개서 히스토그램 그려라.  
 hist(calories,breaks=100)



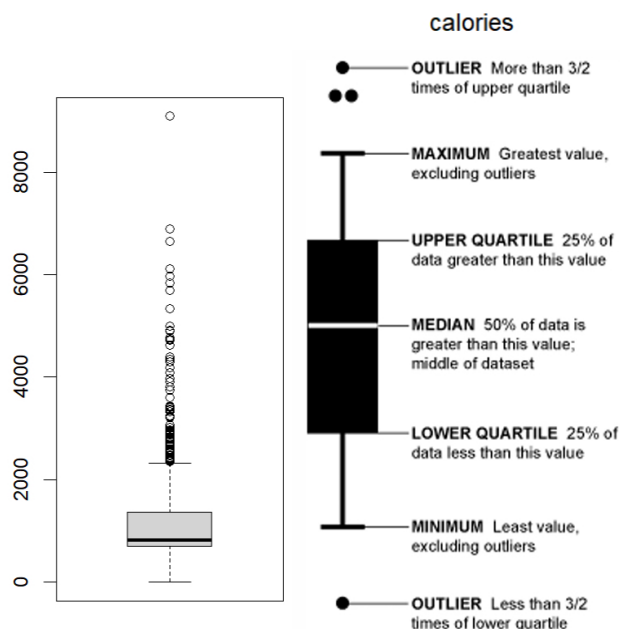
### boxplot()

박스플롯을 그림.  
 boxplot(calories)

MAXIMUM : outlier를 제외한 최고값  
 UPPER QUARTILE : 선보다 위에 25퍼센트의 값이 있음.  
 MEDIAN : 선보다 위에 25퍼센트의 값이 있음.  
 LOWER QUARTILE : 선보다 아래 25퍼센트의 값이 있음.  
 MINIMUM : outlier를 제외한 최솟값

OUTLIER : 박스 밖 선 바깥에 있는 점들은 outlier임.

가운데 선은 중간값(Median)



`boxplot(결과 열이름 ~ 항목 열이름)`

`boxplot`에 `activity` 별로 `calories`들을 그려라!

`boxplot(calories ~ activity)`

-> 오른쪽 `boxplot`을 보고 얻을 수 있는 것.

하이킹이 칼로리를 제일 많이 쓰는구나...

조깅은 사람들이 대부분 비슷한 양의 칼로리는 쓰는구나...

### boxplot 옵션들

`boxplot(calories ~ activity,`

`main="Box plots ny activities",`

`# boxplot 이름`

`xlab="Activity types",`

`#x label`

`ylib="Activity types",`

`#y label`

`ylim=c(0,8000),`

`#y제한`

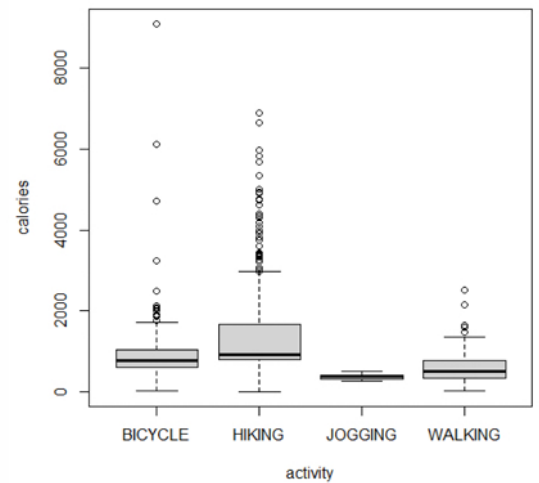
`col= c("blue","red","gray","yellow"))`

`#색깔`

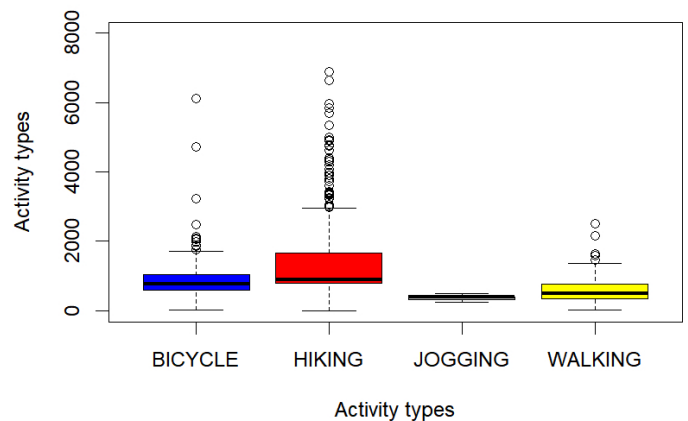
+추가

`boxplot(calories ~ activity+address, ~이하동일`

`#지역별로 나눔`



Box plots ny activities



### 페이지에 여러 plot 표시하기

`par( ) 함수`

`#partition`

`par(mfrow=c(2,1))`

행 단위로 그래프를 배치할 때

첫 열부터 왼->오

`par(mfcol =c(a,b))`

열 단위로 그래프를 배치할 때

첫 행부터 위->아래

e.g. `par(mfrow=c(2,1))`

`#par`을 쓰고 그 이후에 `boxplot`을 쓰면 됨!

`boxplot(calories ~ activity)`

`boxplot(workout_distance ~ activity)`

`par(mfrow=c(2,2))`

`boxplot(calories ~ activity)`

`boxplot(workout_distance ~ activity)`

`boxplot(max_altitude ~ activity)`

`boxplot(avg_vel ~ activity)`

`par(mfcol=c(2,2))`

`boxplot(calories ~ activity)`

`boxplot(workout_distance ~ activity)`

`boxplot(max_altitude ~ activity)`

`boxplot(avg_vel ~ activity)`

### ggplot2을 사용한 그래픽

`ggplot(데이터셋, aes(열이름)) + 그래픽함수()`

그래픽 종류들

`geom_boxplot()` : box plots

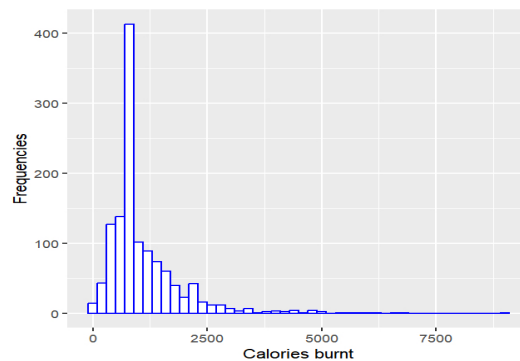
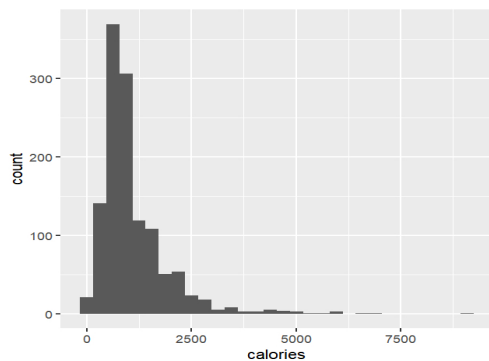
`geom_histogram()` : histogram

`geom_point()` : same as scatter plot

`geom_bar()` : bar plots

e.g. `ggplot(exercise, aes(calories)) + geom_histogram()`

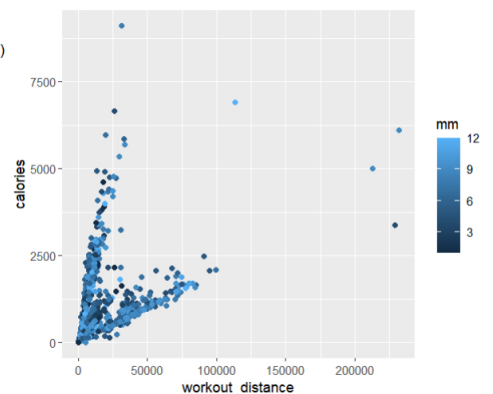
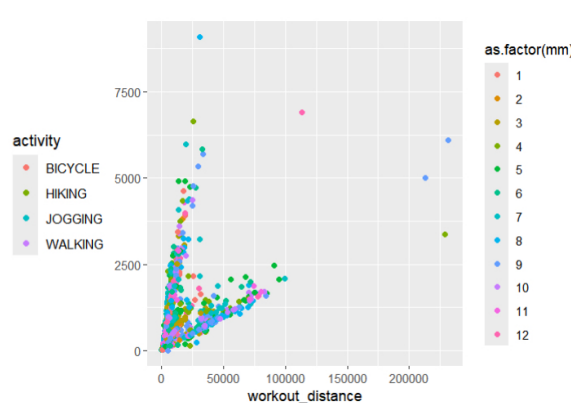
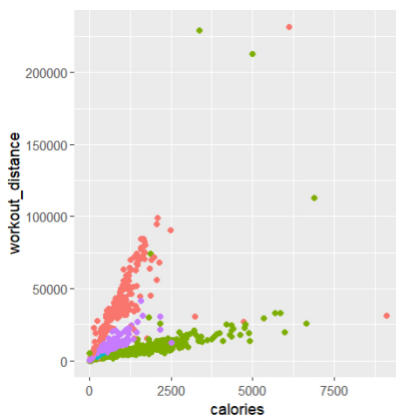
```
ggplot(exercise, aes(calories)) + geom_histogram()
ggplot(exercise, aes(calories)) + geom_histogram(col="blue", fill="white",binwidth=200) +
  xlab("Calories burnt") +
  ylab("Frequencies")
```



### aes() in ggplot()

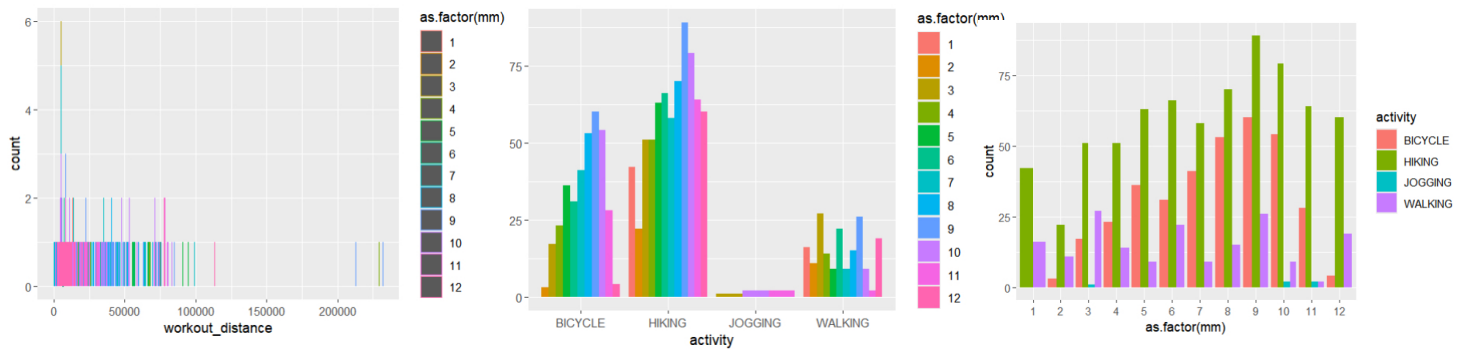
aesthetics은 layer의 모습을 제어함  
 x, y : 사용할 x와 y값  
 color : 테두리 색  
 fill : 색 채우기  
 shape : 모양

```
ggplot(exercise, aes(x=calories, y=workout_distance, color=activity, shape=mm)) + geom_point()
#color를 activity 별로 다르게 하고, shape를 mm별로 다르게 하라는 뜻
ggplot(exercise, aes(y=calories, x=workout_distance, color=as.factor(mm))) + geom_point()
#color를 as.factor를 함. mm열이름을 연속적인 숫자로 보는 것이 아닌, 카테고리로 봄.
ggplot(exercise, aes(y=calories, x=workout_distance, color=mm)) + geom_point()
#color를 as.factor를 함. mm을 연속적인 숫자로 봄
```



```
ggplot(exercise, aes(workout_distance, color=as.factor(mm))) +
  geom_bar(position="stack")
#히스토그램을 그림. 첫 번째 파라미터는 x축. y축은 해당하는 개수를 센 것.
```

```
ggplot(exercise, aes(activity, fill=as.factor(mm))) +
  geom_bar(position="dodge")
#aes
ggplot(exercise, aes(as.factor(mm), fill=activity)) +
  geom_bar(position="dodge")
```



## ggplot\_point

```
ggplot(exercise,aes(x=as.factor(mm), y=calories, color=activity)) +
  geom_point(size=2) + xlab("month") + ylab("calories") + ggtitle("calories and month")
```

```
ggplot(exercise,aes(x=as.factor(mm), y=calories, color=activity)) +
  geom_point(size=2) + xlab("month") + ylab("calories") + ggtitle("calories and month")+
  facet_wrap(~activity)      #특정 열이름으로 그래픽을 분리시킴
```



## Week 9

### FOR loops

무언가를 반복함.

```
for(변수 in
```

```
s = seq(1,100,2)      #s 리스트가 1,3,5 ...로 초기화
s2=NULL               #NULL 로 하든 뭐로 하든 먼저 변수를 선언해야 함. 아니면 에러
for(i in 1:50){
  s2[i] = s[i]^2+3
}
```

### 이중 FOR loop

```
M = matrix(rep(0,9),3,3)      #모든 값이 0인 3x3 matrix
```

```
for(i in 1:3){
  for(j in 1:3){
    M[i,j] = i+j
  }
}
```

### While loop

```
x=0
while(x<10){
  x=x+3
}
```

### if 조건문

```
a=-4
if(a<0){
  a=0
}
```

```
A=c(1,4,-1,4,-2)
N=length(A)
for(i in 1:N){
  if(A[i]>0){
    A[i]=0
  }
}
```

### if, else if, else

```
if(condition 1){
  1이 참이면 실행
}else if(condition 2){ ##주의!!! }으로 시작해야만 함!!!
  1이 거짓이고 2가 참이면 실행
}else{
  1,2가 거짓이면 실행
}      << 중괄호 닫기가 꼭 문장 맨 앞에 와야 함.
```

|                 |         |                |
|-----------------|---------|----------------|
| dummy variable이 | 1개가 있으면 | 2가지 여부 판단 가능   |
|                 | 2개가 있으면 | 3가지 여부 판단 가능   |
|                 | n개가 있으면 | n-1가지 여부 판단 가능 |

```
reg5 <- lm(sales ~ price+adv+disp+mm, data=ad)
```

가격과 광고와 display와 월을 x변수로 함. -> **하지만 이는 잘못됨!!** 왜냐하면 mm(월)을 숫자로 취급해버리면 1월보다 2월이 n만큼 2월보다 3월이 n만큼 더 잘 팔리는 것으로 해석할 수 있음

mm은 **categorical value**가 되어야 한다. factor()나 as.factor()로 바꾸어야 한다.

```
reg5<- lm(sales ~ price+adv+disp+month, data=ad)을 실행했을 때...
```

#month는 JAN, FEB 등...

```
reg6<- lm(sales ~ price+adv+disp+factor(mm), data=ad) 이것도 가능. 숫자를 factor로 바꾸고 난 후 회귀.
```

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )     |
|-------------|----------|------------|---------|--------------|
| (Intercept) | 403.6056 | 13.9861    | 28.858  | < 2e-16 ***  |
| price       | -1.6394  | 0.3874     | -4.232  | 0.000113 *** |
| adv         | 0.5902   | 0.0569     | 10.374  | 1.63e-13 *** |
| disp        | 9.3235   | 2.8202     | 3.306   | 0.001865 **  |
| monthAUG    | 27.0781  | 6.6353     | 4.081   | 0.000181 *** |
| monthDEC    | 7.2978   | 6.4883     | 1.125   | 0.266655     |
| monthFEB    | -5.0109  | 7.0708     | -0.709  | 0.482177     |
| monthJAN    | -4.0894  | 6.5454     | -0.625  | 0.535274     |
| monthJUL    | 19.9461  | 6.3894     | 3.122   | 0.003138 **  |
| monthJUN    | 19.5785  | 6.5545     | 2.987   | 0.004547 **  |
| monthMAR    | -4.1624  | 6.3892     | -0.651  | 0.518049     |
| monthMAY    | 14.1901  | 6.3685     | 2.228   | 0.030910 *   |
| monthNOV    | 6.5347   | 6.3800     | 1.024   | 0.311185     |
| monthOCT    | 10.3056  | 6.6908     | 1.540   | 0.130498     |
| monthSEP    | 17.0738  | 6.4038     | 2.666   | 0.010618 *   |

APR이 없네? APR이 기준인가보다

AUG의 p-value는 0.05보다 작아도 유효함. coef는 27이므로 APR보다 AUG이 27 더 높다

DEC의 p-value는 0.05보다 크니까 유효하지 않음. ==> APR과 별 차이가 없다

### 방정식 세워보기

For **APR**, Sales = 403 - 1.6 price + 0.59 adv + 9.3 disp

For **AUG**, Sales = 403 - 1.6 price + 0.59 adv + 9.3 disp + 27\*1

= (403 + 27) - 1.6 price + 0.59 adv + 9.3 disp      4월보다 8월이 27개 더 많이팔림

### Interaction Effect

두 개 이상의 독립변수가 종속변수에 영향을 미칠 때, 그 효과가 서로 독립적이지 않고 상호 의존적인 경우

$$Sales_t = \beta_0 + \beta_1 \cdot Price_t + \beta_2 \cdot Weekend_t + \beta_3 \cdot Price_t \cdot Weekend_t$$

변수 의미:

1.  $\beta_0$ : 평일(Weekday) 기준, 가격(Price)과 주말 여부(Weekend)가 모두 0일 때의 판매량 (Intercept).
2.  $\beta_1$ : 가격(Price)이 판매량(Sales)에 미치는 기본 효과.
3.  $\beta_2$ : 주말(Weekend)이 판매량(Sales)에 미치는 추가적인 효과(터미 변수로 주말은 1, 평일은 0).
4.  $\beta_3$ : 가격과 주말 간의 **상호작용 효과(interaction effect)**.

```
reg7 <- lm(sales ~ price+adv+disp+price:month, data=ad) #price:month 상호작용항 추가하는 문법
#price가 sales에 미치는 영향이 month에 따라서 달라지는가?
```

```
ad$interaction_term = price * mm #위에랑 이거랑 똑같은!
reg7<- lm(sales ~ price+adv+disp+month + interaction_term, data=ad)
summary(reg7)
```

price:month 이렇게 쓰면 interaction을 나타내는 변수가 됨.

Coefficients:

|                | Estimate  | Std. Error | t value | Pr(> t )     |
|----------------|-----------|------------|---------|--------------|
| (Intercept)    | 419.92216 | 12.88605   | 32.587  | < 2e-16 ***  |
| price          | -2.42482  | 0.58161    | -4.169  | 0.000137 *** |
| adv            | 0.56180   | 0.06049    | 9.288   | 4.99e-12 *** |
| disp           | 7.98343   | 2.89013    | 2.762   | 0.008282 **  |
| price:monthAUG | 2.39055   | 0.68725    | 3.478   | 0.001131 **  |
| price:monthDEC | 0.79423   | 0.55795    | 1.423   | 0.161492     |
| price:monthFEB | -0.59442  | 0.64920    | -0.916  | 0.364750     |
| price:monthJAN | -0.47007  | 0.58206    | -0.808  | 0.423573     |
| price:monthJUL | 1.68110   | 0.57393    | 2.929   | 0.005320 **  |
| price:monthJUN | 1.73090   | 0.65401    | 2.647   | 0.011164 *   |
| price:monthMAR | -0.50900  | 0.63373    | -0.803  | 0.426092     |
| price:monthMAY | 1.17948   | 0.56380    | 2.092   | 0.042110 *   |
| price:monthNOV | 0.54744   | 0.61251    | 0.894   | 0.376204     |
| price:monthOCT | 0.74302   | 0.57519    | 1.292   | 0.203028     |
| price:monthSEP | 1.28804   | 0.62689    | 2.055   | 0.045746 *   |

#### 통계적으로 유의미한 상호작용:

- \*\*AUG (8월)\*\*:  $W(-2.42 + 2.39 = -0.03 \text{ ₩})$   
- 가격 증가가 8월에는 거의 판매량에 영향을 미치지 않습니다.
- \*\*JUL (7월)\*\*:  $W(-2.42 + 1.68 = -0.74 \text{ ₩})$   
- 가격이 증가할 때 판매량이 감소하지만, 그 효과가 4월보다 약해집니다.
- \*\*JUN (6월)\*\*:  $W(-2.42 + 1.73 = -0.69 \text{ ₩})$   
- 가격이 증가할 때 판매량이 감소하지만, 그 효과가 4월보다 약해집니다.
- \*\*MAY (5월)\*\*:  $W(-2.42 + 1.18 = -1.24 \text{ ₩})$   
- 가격의 부정적 영향이 줄어듭니다.
- \*\*SEP (9월)\*\*:  $W(-2.42 + 1.29 = -1.13 \text{ ₩})$   
- 가격 증가로 인한 판매량 감소 폭이 4월보다 작습니다.

## Carry Over effect of Advertising

이월 효과(Carry over effect) -lag model

$N = \text{length}(\text{sales})$

```
reg8<- lm(sales[2:N] ~ price[2:N]+adv[2:N] + adv[1:N-1] +disp[2:N]+month[2:N], data=ad)
```

N-1번째 달의 광고로 N개월의 sales가 어떻게 변했는지 알아보기



```
(Intercept) 342.84406 14.03646 24.425 < 2e-16 ***
price[2:N] -1.66280 0.28811 -5.771 7.86e-07 ***
adv[2:N] 0.62357 0.04215 14.793 < 2e-16 ***
adv[1:N - 1] 0.27147 0.04274 6.352 1.12e-07 ***
```

N=length(sales)

```
reg8<- lm(sales[2:N] ~ price[2:N]+adv[2:N] + adv[1:N-1] +disp[2:N]+month[2:N], data=ad)
```

```
summary(reg8)
```

Logistic Regression 로지스틱 모델 : 0과 1로 결과를 낼 때 적합.

0 구매하지 않음 1 구매함.

#logistic regression 기본 문법

```
logistic.reg<-glm(conversion ~ free_item+sms+email, data=data, family="binomial")
```

```

#(Intercept) -3.6886 0.5736 -6.431 1.27e-10 ***
# free_item 0.5031 0.2042 2.463 0.0138 *
# sms 0.7768 0.3466 2.241 0.0250 *
# email 0.2488 0.5611 0.443 0.6574

```

#통계적 의미 O free\_item이 늘면 구매가 늘.  
#통계적 의미 O sms가 늘면 구매가 늘.  
#통계적 의미 X

Step1,2는 그대로 할 수 있음.

$P(\text{conversion}=1) = \exp(-3.6 + 0.5 \cdot \text{free\_item}) / (1 + \exp(-3.6 + 0.5 \cdot \text{free\_item}))$  이 식에 넣어봅시다..

```
#free_item=0
```

```
p0 = exp(-3.6+0.5*0)/(1+exp(-3.6+0.5*0))
```

```
p0 #==0.0265
```

```
#free_item=1
```

```
p1 = exp(-3.6+0.5*1)/(1+exp(-3.6+0.5*1))
```

```
p1 #==0.043 (1.65% or 0.0165 증가)
```

```
#free_item=2
```

```
p2 = exp(-3.6+0.5*2)/(1+exp(-3.6+0.5*2))
```

```
p2 #0.0691 (2.61% or 0.0261 증가)
```

Poisson Model 포아송 모델 : 데이터가 음수가 아니며(0 이상의 정수), 사건의 횟수일 때

$$\log(\mu) = b_0 + b_1X_1 + \dots + b_kX_k$$

```
hist(visit, breaks=50)
```

```
poisson.reg<-glm(visit ~ free_item+sms+email, data=data, family="poisson")
```

Coefficients:

```

              Estimate Std. Error z value Pr(>|z|)
(Intercept) -3.56018    0.32199 -11.057 < 2e-16 ***
free_item    0.99621    0.06045  16.479 < 2e-16 ***
sms          0.60823    0.17466   3.482 0.000497 ***
email        0.57244    0.24124   2.373 0.017647 *
```

```
#log(mu) = -3.56+0.6*sms
```

```
#mu = exp(-3.56+0.6*sms)
```

```
mu0 = exp(-3.56+0.6*0) #0.02
```

```
mu1 = exp(-3.56+0.6*1) #0.05
```

```
mu2 = exp(-3.56+0.6*2) #0.09
```

#문자를 1만큼 늘리면 0.03이 늘어난다

#문자를 2만큼 늘리면 0.04만큼 늘어난다.

```
mu0= exp(-3.56+ 0.6*x) = exp(-3.56)*exp(0.6*x)
mu1= exp(-3.56+ 0.6*(x+1)) = exp(-3.56)*exp(0.6*x)*exp(0.6)
mu1/mu0 = exp(0.6) ~1.82 #나누면 소거됨
mu1= 1.82*mu0
# x가 1 늘어나면, y는 82% 증가한다.
```

log linear model 로그 선형 모델 :

```
ln.visit = log(visit)
#log(0)은 마이너스 무한이라 -Inf가 많이 나옴.
data$ln.visit = log(visit+1) #그래서 이런 식으로 고..?
linear3.reg<-lm(ln.visit ~ free_item+sms+email, data=data)
```

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t ) |     |
|-------------|----------|------------|---------|----------|-----|
| (Intercept) | -0.13644 | 0.03896    | -3.502  | 0.000571 | *** |
| free_item   | 0.22481  | 0.02001    | 11.236  | < 2e-16  | *** |
| sms         | 0.10743  | 0.03417    | 3.144   | 0.001924 | **  |
| email       | 0.10227  | 0.04778    | 2.141   | 0.033541 | *   |

#ggpredict 사용하기

```
reg1 <- lm(sales ~ price+adv+disp, data=ad)
pred <- ggpredict(reg1, terms="price")
```

| price | Predicted | 95% CI      |
|-------|-----------|-------------|
|       | #예측값      | #95프로 신뢰 구간 |

```
-----
3 | 529.41 | 519.70, 539.11
6 | 523.33 | 516.15, 530.50
8 | 519.27 | 513.49, 525.06
10 | 515.22 | 510.33, 520.11
11 | 513.19 | 508.48, 517.91
13 | 509.14 | 504.12, 514.16
15 | 505.09 | 499.04, 511.13
19 | 496.98 | 487.79, 506.17
```

>>결과는 항상 범위로 나옴

만약 이렇게 두 개로 결과가 나왔다면,

```
prediction 1) 520 [525,535]
```

```
prediction 2) 520 [500,540]
```

1이 2보다 더 reliable하다고 할 수 있음. >> 더 좁아야 reliable, accurate함!

#dplyr 사용하기

```
lm(sales ~ price+adv+disp, data=ad)
```

```
ggpredict(reg1, terms="price")
```

이걸 합쳐서 파이프코드로 만들어버리자!

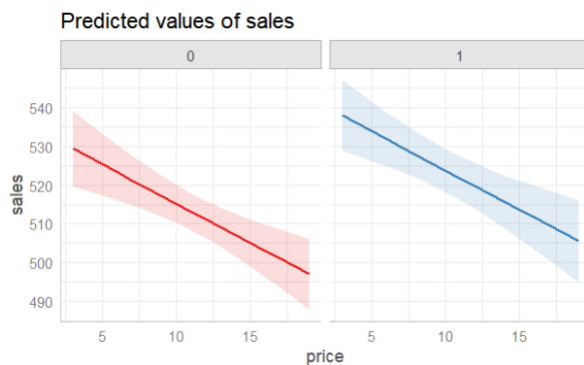
```
lm(sales ~ price+adv+disp, data=ad) %>%
  ggpredict(terms="price") %>% #pipe에서는 첫 인자 없애기
  plot()
```



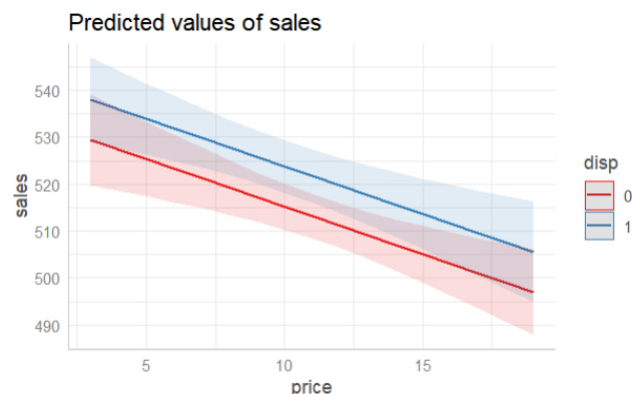
```
lm(sales ~ price+adv+disp, data=ad) %>%
  ggpredict(terms=c("price","disp")) %>% #pipe에서는 첫 인자 없애기
  plot(facet=TRUE) #여러개 따로 표현함
```

또는

```
plot(facet=FALSE) 또는 파라미터 없음 #여러개 동시에 표현함
```



facet==TRUE



## #pander 패키지

```
reg1 <- lm(sales ~ price+adv+disp, data=ad)
pander(reg1)
```

|                 | Estimate | Std. Error | t value | Pr(> t )  |
|-----------------|----------|------------|---------|-----------|
| **(Intercept)** | 412.4    | 14.51      | 28.43   | 5.793e-35 |
| **price**       | -2.027   | 0.5109     | -3.967  | 0.0002091 |
| **adv**         | 0.614    | 0.06762    | 9.081   | 1.336e-12 |
| **disp**        | 8.539    | 3.698      | 2.309   | 0.02466   |

#huxtable 패키지 여러 개 reg를 한눈에 볼 수 있음.

```
reg1 <- lm(sales ~ price, data=ad)
reg2 <- lm(sales ~ price+adv, data=ad)
reg3 <- lm(sales ~ price+adv+disp, data=ad)
huxreg(reg1,reg2,reg3)
huxreg("base model"=reg1 , "model1" = reg2, "model2"=reg3) #결과 확인해보기
```

|             | base        | dddel | model1      | model2      |
|-------------|-------------|-------|-------------|-------------|
| (Intercept) | 535.532 *** |       | 420.552 *** | 412.381 *** |
|             | (8.838)     |       | (14.591)    | (14.505)    |
| price       | -1.638 *    |       | -2.232 ***  | -2.027 ***  |
|             | (0.778)     |       | (0.522)     | (0.511)     |
| adv         |             |       | 0.602 ***   | 0.614 ***   |
|             |             |       | (0.070)     | (0.068)     |

|      |  |  |         |
|------|--|--|---------|
| disp |  |  | 8.539 * |
|      |  |  | (3.698) |

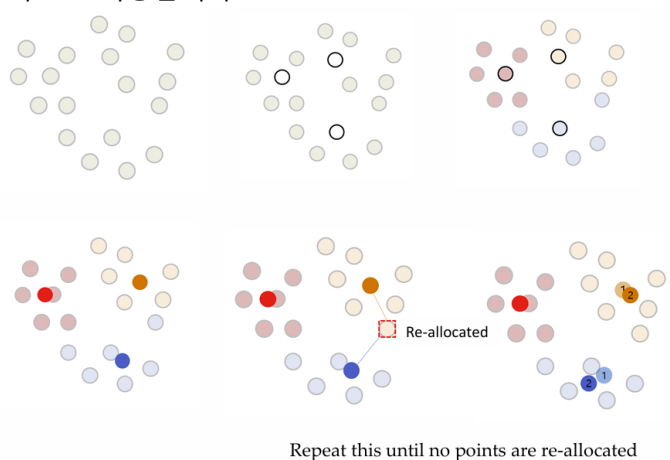
|        |          |          |          |
|--------|----------|----------|----------|
| N      | 60       | 60       | 60       |
| R2     | 0.071    | 0.596    | 0.631    |
| logLik | -268.443 | -243.450 | -240.722 |
| AIC    | 542.885  | 494.899  | 491.443  |

\*\*\* p < 0.001; \*\* p < 0.01; \* p < 0.05.

Column names: names, base dddel, model1, model2

## K-means Clustering (비지도학습)

K-means 클러스터링은 데이터를 K개의 클러스터로 나누는 비계층적 군집화(flat clustering) 알고리즘입니다. 데이터를 클러스터로 나누기 위해 중심점(centroid, 중심)을 활용하며, 각 데이터가 가장 가까운 중심점에 할당되는 방식으로 작동합니다.



`ab1 <- ab[,c(10,12)]`      가격과 리뷰수 column만 선택

`kmeans(data, centers=클러스터의 수)`

`ab_kmeans <- kmeans(ab1,centers=4)`

#비지도학습이라 cluster의 개수를 정해줘야 함!

`group=ab_kmeans$cluster`      #ab means 중 클러스터만 보여줘

`ab_kmeans$centers`      #ab means 중 센터의 값들만 보여줘

| #price      | number_of_reviews |
|-------------|-------------------|
| #1 140.7864 | 36.83172          |
| #2 117.0301 | 176.16541         |
| #3 114.6774 | 358.83871         |
| #4 517.1111 | 38.11111          |

## #시각화하기 by ggplot2 - qplot

`ab1$cluster <- as.factor(ab_kmeans$cluster)`

시각화를 해보자! - `qplot(x축 열이름, y축 열이름, colour=열이름(클러스터), data=데이터셋)`

`ab1$cluster <- as.factor(ab_kmeans$cluster)` #factor로 바꾸기

`qplot(price, number_of_reviews, colour=cluster, data=ab1)`

#맨해튼에 있는것만 고르기

```
Manhattan_kmeans <- ab %>% filter(neighbourhood_group=="Manhattan") %>% #행 sort
select(price, number_of_reviews) %>% #열 sort
kmeans(centers=4)
```

```
Manhattan_data <- ab %>% filter(neighbourhood_group=="Manhattan") %>%
select(price, number_of_reviews) # 가격과 리뷰 행을 모은 데이터셋
Manhattan_data$cluster <- as.factor(Manhattan_kmeans$cluster) #거기에 cluster번호 추가! 팩터로.
```

```
qplot(price, number_of_reviews, colour=cluster, data=Manhattan_data) #qplot으로 그리기
```

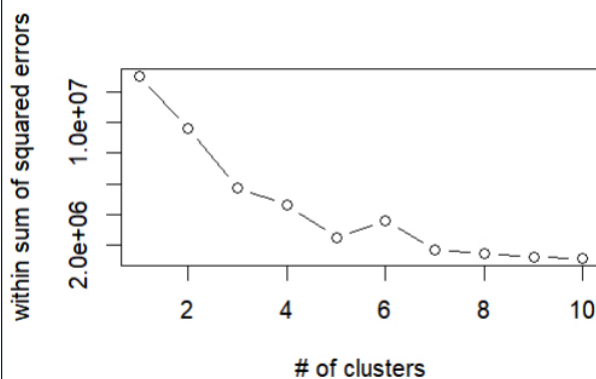
#최적의 클러스터 숫자는?

```
ab_kmeans <- kmeans(ab1, centers=4) #여기서 centers 숫자를 2,3,4,5,6 등으로 바꿔보자
ab_kmeans$withinss
```

[1] 223071.0 2634575.9 1497187.7 685556.6 #결과가 나오는데, 이 숫자의 합이 작을수록 좋은 클러스터 수!

#몇 개가 좋은지 찾아보자!

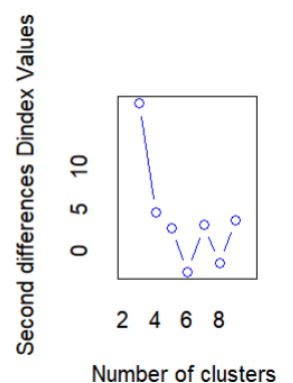
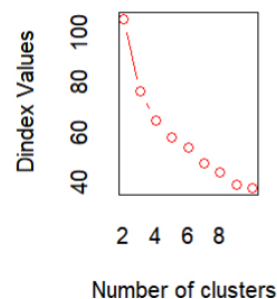
```
ws = c() #빈 array를 미리 선언해줘야 에러가 안남
for (i in 1:10){
  ab_kmeans <- kmeans(ab1, centers=i)
  ws[i]=sum(ab_kmeans$withinss) #sum이 작을수록 더 나은 clustering 분석이다.
}
```



#3과 7이 elbow point라고 불림. 거기서 멈추는 것이 좋음. 급격하게 변하는 지점.

#NbClust

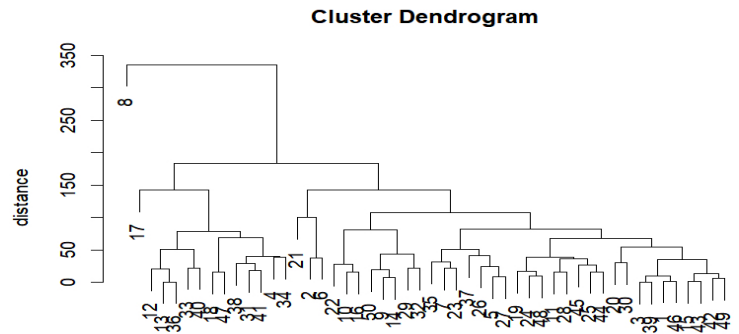
```
ab1 <- ab[c(10,12)] #가격과 리뷰수 행 선택
NbClust(데이터셋, min.nc=최소, max.nc=최대, method="kmeans")
nc <- NbClust(ab1, min.nc=2, max.nc=10, method="kmeans")
```



## #Hierarchical Clustering (비지도학습)

Hierarchical Clustering은 데이터를 계층적으로 그룹화하여 군집화하는 방법입니다. 이 과정에서 데이터 간의 유사성을 기반으로 클러스터를 병합하거나 분할하며, 최종적으로 \*\*덴드로그램(dendrogram)\*\*이라는 트리 구조를 생성합니다.

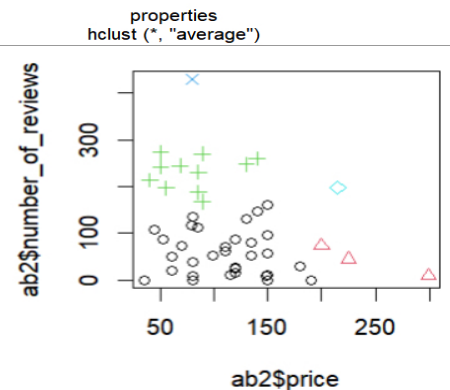
```
dedogram <- hclust(dist(ab2), method="average")
plot(dedogram, xlab="properties", ylab="distance")
```



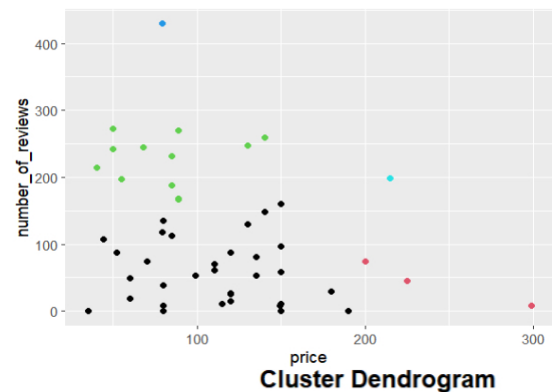
```
cluster_hc<- cutree(dedogram, 5)
```

#덴도그램을 잘라서 5개로 나누어라!

```
plot(ab2$price, ab2$number_of_reviews, col=cluster_hc, pch=cluster_hc)
#col이랑 pch는 색깔과 모양 바꾸기
```



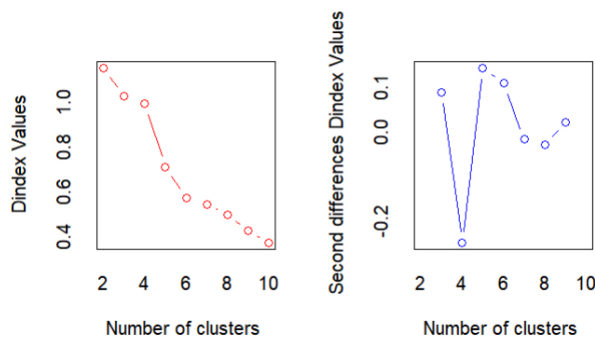
```
ggplot(ab2, aes(price, number_of_reviews)) +
  geom_point(col=cluster_hc) +
  scale_color_manual(values = c("black","red","green"))
```



```
plot(dedogram, xlab="properties", ylab="distance")
```

```
rect.hclust(dedogram,k=4)
```

#dedogram을 4개로 나누어 사각형을 그려라



```
ab2_scaled <- scale(ab2)
```

#표준화...? 데이터간 차이가 클러스터링에 영향을 미치지 않도록 처리

```
nc <- NbClust(ab2_scaled, min.nc=2, max.nc=10, distance="euclidean", method="average")
```

## #Supervised 지도학습

### Decision Tree 선택 트리

트레이닝 데이터(고객들의 정보 등..)를 통해서 결과를 도출함.

트레이닝 데이터에는 x와 y가 있음.

새로운 고객이 살지 안살지를 예측.

사람이 어떻게 판단하는지를 모방함

Purity == 얼마나 같은가? homogenous한가? 순수한가? 섞여있지 않는가?

**이 값이 작을수록 pure함.**

식을 외울 필요는 X

정확히 반반이면 purity가 제일 높음.

Gini index(I) 의 최솟값(제일 덜 순수함, 고루 섞임)은 0.5 최댓값(제일 순수한 값, 하나로만 이루어짐)은 0

선으로 전체 데이터를 이등분했을 때 점점 더 purity가 낮아지면 잘 분리되고 있는 것!

$$I = \sum_{i=1}^d \left( R_i \left( 1 - \sum_{k=1}^m (P_{ik})^2 \right) \right)$$

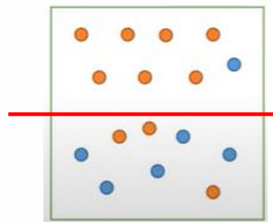
$$I = 0.5 \times (1 - (7/8)^2 - (1/8)^2)$$

$$+ 0.5 \times (1 - (3/8)^2 - (5/8)^2)$$

$$= 0.34$$

• Information gain after splitting

$$\bullet 0.47 - 0.34 = 0.13$$



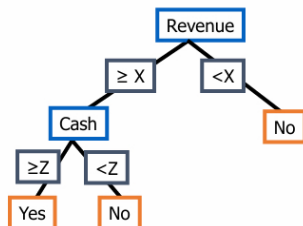
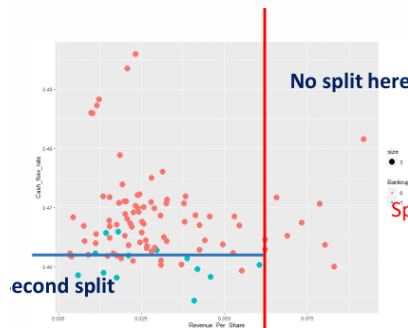
### #Entropy 엔트로피

얼마나 pure한가?

개념적으로는 purrity와 같지만

공식이 다름.

purtiy 대신 사용!



| Revenue_Per_Share | Cash_flow_rate | Bankrupt |
|-------------------|----------------|----------|
| 0.00348           | 0.46220        | 0        |
| 0.00594           | 0.45852        | 1        |
| 0.01119           | 0.46224        | 1        |
| 0.01142           | 0.46549        | 0        |
| 0.01227           | 0.48824        | 0        |
| 0.01437           | 0.46571        | 1        |
| 0.01553           | 0.46482        | 0        |
| 0.01756           | 0.45814        | 1        |
| 0.01810           | 0.46586        | 1        |
| 0.01859           | 0.47877        | 0        |
| 0.02006           | 0.46516        | 0        |
| 0.02114           | 0.46187        | 1        |
| 0.02382           | 0.47209        | 0        |
| 0.02464           | 0.46410        | 0        |
| 0.02571           | 0.46720        | 0        |
| 0.02576           | 0.47006        | 0        |
| 0.02660           | 0.46314        | 0        |
| 0.02969           | 0.46275        | 1        |
| 0.03091           | 0.46102        | 0        |
| 0.03886           | 0.46138        | 1        |
| 0.04256           | 0.46702        | 0        |
| 0.05298           | 0.46844        | 0        |

### Over-fitting 오버피팅

훈련 데이터에 너무 지나치게 맞춰져서, 새로운 데이터에서는 일반화를 잘 하지 못하는 현상

오버피팅을 피하기 위해서? -> pruning

일반적으로 트리는 자세할수록(클수록) 좋지만, 테스트 데이터를 구분하지 못할 수 있음

### Pruning 가지치기

트레이닝에 없는 테스트 데이터가 들어오면? -> 오버피팅 문제가 생긴다!

트레이닝에 있는 노란색(밝은색)을 없애버린다!(Pruning). 더 나은 결과값을 냄.

tree 전체(full tree)를 사용하기보다 일부 branch를 없애버리는게 더 나은 결과를 낼 수 있다.

Pruning에 따라서 트리가 다양하게 나올 수 있다.

가지치기를 통해 나무를 단순화하면 훈련 데이터에서의 오류율은 증가할 수 있지만, 테스트 데이터에서의 오류율은 줄어든 가능성이 있음.

## Regression Tree

k-means과 원리가 동일함.

Decision tree와 비슷하지만 purity 대신 SSE(Sum of Squared Error)를 사용한다. 빼서 제곱!  
SSE가 제일 작은 것을 기준으로 한다.

Decision Tree와 동일한 방식으로 작동하지만, 목표는 연속형 변수를 예측하는 것입니다

## #rpart Decision Tree 생성

rpart() 함수는 간단하게 **Decision Tree** 모델을 생성할 수 있는 강력한 도구입니다.

```
rpart(y값 ~ x값, data = )
```

```
dt_car <- rpart(choice~., data=car) # y 값은 choice, x값은 choice를 제외한 행 전부
```

#predict() 모델을 사용해 데이터프레임에 대한 y 값 예측.. 0, 1 등으로 반환

predict(**학습된 decision tree 모델**, newdata= **테스트할 데이터프레임**, type="class" yes/no로 반환)

```
pred_car <- predict(dt_car, newdata=car, type="class")
```

```
car$choice == pred_car #이러면 car$choice에 yes나 no 값이 저장된다.
```

```
accuracy_rate <- sum(car$choice == pred_car)/nrow(car) #sum을 쓰면 ture를 1로, false를 0으로 센다.
```

```
# accuracy_rate
```

```
#[1] 0.7211667
```

e.g.

```
sp <- read.csv("spotify.csv")
```

```
attach(sp)
```

```
dt_sp <- rpart(genre ~., data=sp)
```

```
plot(dt_sp)
```

```
text(dt_sp)
```

```
pred_sp <- predict(dt_sp, newdata=sp, type="class")
```

```
accuracy_rate = sum(sp$genre==pred_sp) / nrow(sp) #[1] 0.7330827
```

e.g.

```
br <- read.csv("bankrupt.csv")
```

```
attach(br)
```

```
N = nrow(br) #총 몇줄있는가?
```

```
br$Bankrupt = as.factor(br$Bankrupt) #값. 파산할까?(1) or 아닐까?(0) factor로 변환
```

```
# 70퍼센트로 training 만들고 30퍼센트로 test를 만들자!
```

```
train <- sample(1:N, round(0.7*N), FALSE) #sample N*0.7개만큼 랜덤뽑기. FASLE 는 뽑은거 또 못뽑게.
```

```
br_train <- br[train, ] #train에서 뽑은 거를 train으로. 콤마 뒤 비어두면 모든열 선택
```

```
br_test = br[-train,] #-를 하면 train에서 나온 숫자만 빼고 나머지를 test로.
```



```
dt1 <- rpart(Bankrupt~., data=br_train)
```

```
dt1_pred <- predict(dt1, newdata=br_test, type="class")
```

```
dt1_pred == br_test$Bankrupt
```

```
ac1_rate <- mean(dt1_pred == br_test$Bankrupt) #mean을 사용하면 간단하게 평균을 낼 수 있다.
```

단순히 Accuracy만 보면 안되고 confusion matrix도 봐야 함.

#### #Confusion Matrix

```
confusionMatrix(예측값, 실제값)
```

```
confusionMatrix(dt1_pred, br_test$Bankrupt)
```

|             | Reference(실제) |         |               |               |
|-------------|---------------|---------|---------------|---------------|
| #Prediction | 0             | 1       |               |               |
| #           | 0 1953(TN)    | 54 (FN) | TrueNegative  | FalseNegative |
| #           | 1 28(FP)      | 11 (TP) | FalsePositive | TruePositive  |

TN : Negative라고 생각했는데 실제로는 틀렸네?

FN : 틀렸다고 생각했는데(Pred=0) 실제로는 맞았네(Ref=1)?

FP : 맞았다고 생각했는데(pred=1) 실제로는 틀렸네(ref=0)?

TP : Positive라고 생각했는데 실제로는 맞았네?

Reference : 실제로 틀렸는가? vs 실제로 맞았는가?

Prediction : Positive로 판단했는가? Negative로 판단했는가?

#1 Right for predicicing 0 (wrong for predicting 0)

-> high TN, low PN

#2)Right for predicicing 1 (wrong for predicting 1)

-> high TP, low FP

=>언제 어떤 모델을 올라야 할까?

1) 머신러닝 모델을 이용해 구매할 가능성이 높은 고객에게 10\$ 쿠폰을 주는 상황.

FN : 구매 안할 것으로 예측돼 쿠폰을 안 보냈지만, 구매를 함

FP : 구매할 것으로 예측돼 쿠폰을 보냈지만, 구매를 안함.

FP에서는 기업 입장에서 10달러를 버림

이 시나리오에서는 FP가 낮은 모델을 골라야 함.

2) 코로나 19 검사

FN : 이 사람이 안 걸릴 것으로 예측됐지만, 실제로 감염됨! -> 관측되지 않은 감염자... 위험그잡채

FP : 이 사람이 걸릴 것으로 예측됐지만, 실제로 감염되지 않음! ->격리하게 해서 미안합니당...

이 시나리오에서는 FN이 낮은 것이 좋다!

#rpart.plot      decision tree 시각화

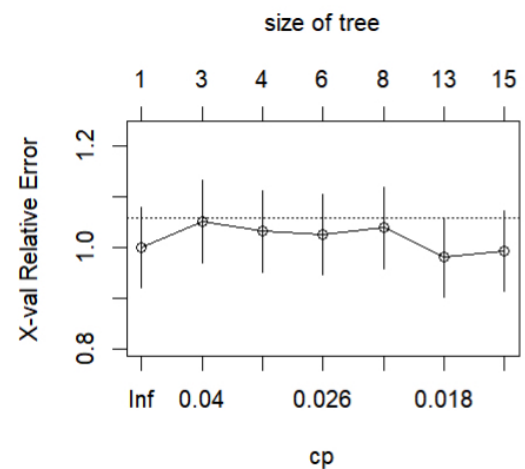
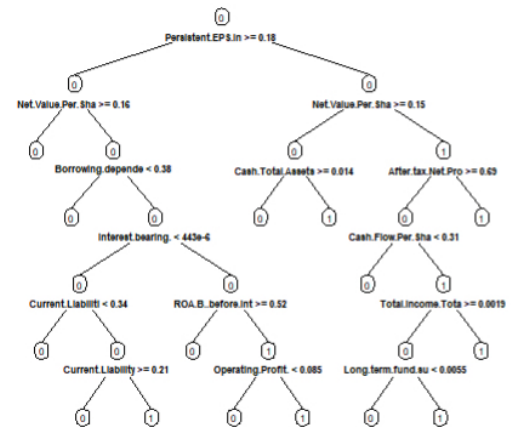
prp(dt1, type=2, extra=0, under=T, yesno=F)

#prp(데이터셋, 그래프 타입, 추가정보, 아래표시?, y/n표시?)

#pruning 하기

dt1\$cptable      #cptable은 복잡도 매트릭스를 저장하는 요소  
plotcp(dt1)

>>여기서 최솟값을 prune하는데 cp값으로 사용!



dt1\_prune1 <- prune(dt1, cp=0.018)

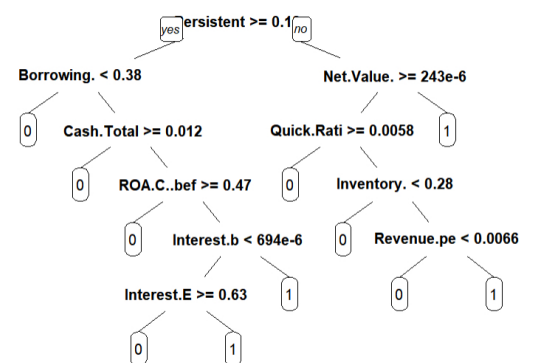
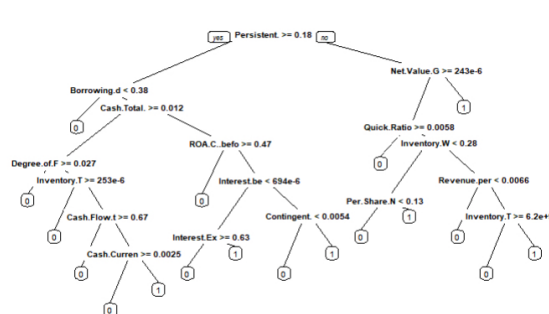
#cp : 가지치기의 기준. 각 분기에서 0.018보다 작으면 해당 분기 제거

par(mfrow=c(2,1))

prp(dt1)

prp(dt1\_prune1)

dev.off()



#pruned tree

# 0 1969 57

# 1 15 5

#original tree

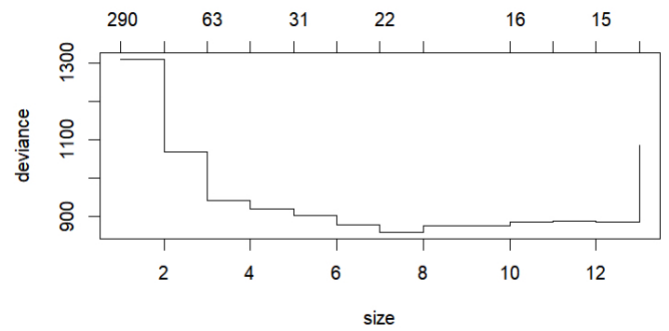
#0 1953 51

#1 31 11

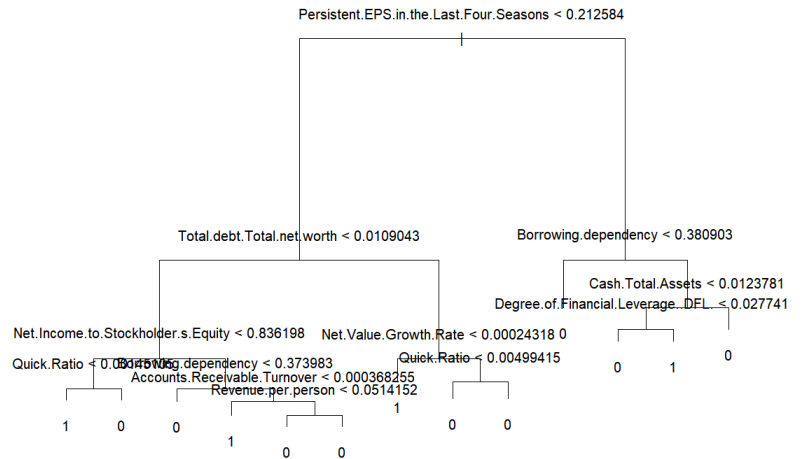
## #tree() 패키지

```
dt2 <- tree(Bankrupt~., data=br_train)
```

```
cvtree <- cv.tree(dt2,k=NULL, best=NULL) #best는 트리 크기
plot(cvtree)
```



```
dt_prune2 <- prune.tree(dt2, best=8) #prune tree는 tree라이브러리의 일부임! r에 포함된 함수가 아님.
plot(dt2)
text(dt2)
```



```
dt_prune_pred2 <- predict(dt_prune2, newdata=br_test, type="class")
table(dt_prune_pred2, br_test$Bankrupt)
```

실제값

| dt_prune_pred2 | 0    | 1  |
|----------------|------|----|
| 0              | 1971 | 73 |
| 1              | 1    | 1  |

## #Neural Network

### #nnnet()

```
car$choice <- factor(car$choice)
train <- sample(1:nrow(car), round(0.7*nrow(car)),FALSE)
car_train = car[train,]
car_test = car[-train,]
```

```
nn1 <- nnet(choice ~ .,data=car_train, size=20, maxit=1000) #size는 히든 레이어 개수, maxit는 최대 반복횟수
nn1_pred <- predict(nn1,newdata=car_test, type="class")
```

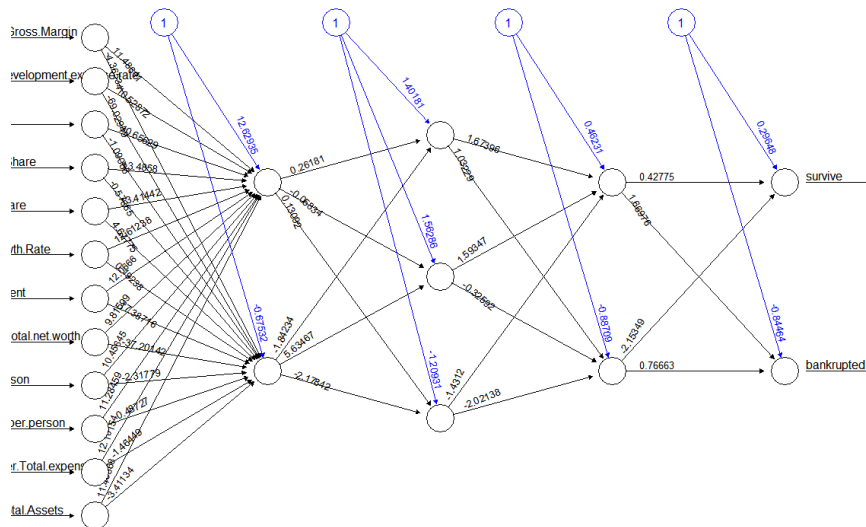
```
confusionMatrix(as.factor(nn1_pred),car_test$choice)
```

| #           |          | Reference |
|-------------|----------|-----------|
| #Prediction | 0        | 1         |
| # 0         | 1030(TN) | 306(FN)   |
| # 1         | 157 (FP) | 307(TP)   |

### #neuralnet() - visualize 쉬움

neuralnet은 다른 함수처럼 ~.을 지원하지 않아서 fomula에 따로 만들어서 넣어놓음.

```
nn2 <- neuralnet(formula, data=br_train, hidden=c(2,3,2), act.fct = "logistic",stepmax=10000)
plot(nn2)
```



```
nn2_pred <- predict(nn2,newdata=br_test, type="class")
```